

IF2240 Basdat

## SQL (Part 1-5)

[SQL Parts](#)

[Basic Query Structure](#)

[SELECT](#)

[WHERE](#)

[WHERE Clause Predicates](#)

[FROM](#)

["Self Join"](#)

[String Operations](#)

[Ordering the Display of Tuples](#)

[SET OPERATIONS](#)

[Null Values](#)

[Aggregate Functions](#)

[GROUP BY](#)

[HAVING](#)

[Joined Relations](#)

[Natural Join](#)

[Inner Join](#)

[Outer Join](#)

[Left Outer Join](#)

[Right Outer Join](#)

[Full Outer Join](#)

[Joined Types and Conditions](#)

[Nested Subqueries](#)

[Nested Query in WHERE Clause](#)

[Set Membership](#)

[Set Comparison](#)

[SOME Clause](#)

[ALL Clause](#)

[Test for Empty Relations](#)

[EXISTS Clause](#)

[NOT EXISTS Clause](#)

[Test for Absence of Duplicate Tuples](#)

[Nested Queries in the FROM Clause](#)

[WITH Clause](#)

[Scalar Subquery](#)

[VIEW](#)

[View Defined using Other Views](#)

[Materialized Views](#)

[Modification of the Database](#)

[Deletion](#)

[Insertion](#)

[Updating](#)

[CASE Statement for Conditional Updates](#)

[Updates with Scalar Subqueries](#)

[Update of a View](#)

[Data Definition Language](#)

[Domain Types](#)

[CREATE TABLE](#)

[Integrity Constraints](#)

[Updates to Tables](#)

[Drop Table](#)

[Alter](#)

[Latihan](#)

## SQL Parts

- DML provides the ability to query information from the database and to insert tuples into, delete tuples from, and modify tuples in the database.
- Integrity
- View definition  
DDL (Data definition language) urusannya dengan skema (berurusan dengan integritas dan view definition)
- Transaction control
- Embedded SQL and dynamic SQL
- Authorization

## Basic Query Structure

A typical SQL query has the form:

```
SELECT  $A_1, A_2, \dots, A_n$   
FROM  $r_1, r_2, \dots, r_m$   
WHERE  $P$ 
```

- $A_i$  represents an attribute
- $R_i$  represents a relation
- $P$  is a predicate

The result of an SQL query is a relation

## SELECT

- The select clause lists the **attributes desired** in the result of a query, which corresponds to the projection operation of the relational algebra

- For example: find all the names of the instructors:

```
SELECT name  
FROM instructor
```

- **Note:** SQL names are case insensitive
- Keyword DISTINCT eliminates duplicates
- Keyword ALL specifies that duplicates should not be removed
- An asterisk (\*) denotes **all attributes**
- Select can contain arithmetic expressions, involving the operation +, -, \*, / and operating on constants or attributes of tuples
- For example:

```
SELECT ID, name, salary/12  
FROM instructor
```

would return a relation that is the same as the instructor relation except that the value of attribute salary is divided by 12.

- Keyword AS is used to rename.

## WHERE

- The WHERE clause specifies conditions that the result must satisfy
- For example:

```
SELECT name  
FROM instructor  
WHERE dept_name = 'Comp. Sci.'
```

## WHERE Clause Predicates

- BETWEEN comparison operator

- For example: Find the names of all instructors with salary between \$90,000 and \$100,000 ( $\geq \$90,000$  AND  $\leq \$100,000$ )

```
SELECT name
FROM instructor
WHERE salary BETWEEN 90000 AND 100000
```

- Tuple comparison

- For example:

```
SELECT name, course_id
FROM instructor, teaches
WHERE (instructor.ID, dept_name) = (teaches.ID, 'Biology')
```

## FROM

- The FROM clause lists the relations involved in the query, which corresponds to the cartesian product operation in relational algebra.
- For example: Find the names of all instructors who have taught some course and the course\_id

```
SELECT name, course_id
FROM instructor, teaches
WHERE instructor.ID = teaches.ID
```

## "Self Join"

Relation *emp-super*

| <i>person</i> | <i>supervisor</i> |
|---------------|-------------------|
| Bob           | Alice             |
| Mary          | Susan             |
| Alice         | David             |
| David         | Mary              |

Find the supervisor of "Bob"

Find the supervisor of the supervisor of "Bob"

Can you find ALL the supervisors (direct and indirect) of "Bob"?

- Supervisor of Bob:

```
SELECT supervisor
FROM emp-super
WHERE person = 'Bob';
```

- Supervisor of supervisor of Bob ("Self Join")

```
SELECT e2.supervisor
FROM emp_super e1, emp_super e2
WHERE e1.supervisor = e2.person
AND e1.person = 'Bob';
```

- Find all supervisors (recursive query)

```
WITH RECURSIVE supervisors AS (
    SELECT person, supervisor
    FROM emp_super
    WHERE person = 'Bob'
    UNION ALL
    SELECT e.person, e.supervisor
    FROM emp_super e
```

```
JOIN supervisors s ON e.person = s.supervisor
)
SELECT supervisor
FROM supervisors;
```

## String Operations

- The operator LIKE uses patterns that are described using two special characters:
  - % character matches any substring
  - \_ character matches any character
  - For example:

Find the names of all instructors whose name includes the substring "dar"

```
SELECT name
FROM instructor
WHERE name LIKE '%dar%'
```

Match the string "100%"

```
LIKE '100\%' ESCAPE '\'
```

we use backslash (\) as the escape character.

- Patterns are case sensitive
  - Example of pattern matching:
    - 'Intro%' matches any string beginning with 'Intro'
    - '%Comp%' matches any string containing 'Comp' as a substring
    - ' \_ \_ ' matches any string of exactly three characters
    - ' \_ \_ \_ %' matches any string of at least three characters
- Concatenation: use "||"

- Conversion from upper to lower case
- Finding string length, extracting substrings, etc.

## Ordering the Display of Tuples

- usage: ORDER BY ... ASC/DESC
- The default is ASC (if you don't specify it will be set to default)
- For example: list in alphabetic order the names of all instructors

```
SELECT DISTINCT name
FROM instructor
ORDER BY name DESC
```

## SET OPERATIONS

- Set operations: **UNION**, **INTERSECT**, **EXCEPT** → each automatically eliminates duplicates
- To retain all duplicates, use the keyword ALL after the operation (UNION ALL, INTERSECT ALL, EXCEPT ALL)
- Examples:
  - Find courses that ran in Fall 2017 or in Spring 2018

```
(SELECT course_id
FROM section
WHERE sem = 'Fall' AND year = 2017)
UNION
(SELECT course_id
FROM section
WHERE sem = 'Spring' AND year = 2018)
```



```
WHERE sem = 'Spring' AND year = 2018)
```

- Find courses that ran in Fall 2017 and in Spring 2018

```
(SELECT course_id  
FROM section  
WHERE sem = 'Fall' AND year = 2017)  
  
INTERSECT  
  
(SELECT course_id  
FROM section  
WHERE sem = 'Spring' AND year = 2018)
```

- Find courses that ran in Fall 2017 but not in Spring 2018

```
(SELECT course_id  
FROM section  
WHERE sem = 'Fall' AND year = 2017)  
  
EXCEPT  
  
(SELECT course_id  
FROM section  
WHERE sem = 'Spring' AND year = 2018)
```

## Null Values

- **null** signifies an unknown value or that a value does not exist
- The result of any arithmetic expressions involving null is null ( $5 + \text{null} = \text{null}$ )
- For example:

Find all instructors whose salary is null

```
SELECT name  
FROM instructor  
WHERE salary IS NULL
```

- Other than predicates IS NULL and IS NOT NULL, SQL treats the result as any comparison involving a null value a **unknown**.
- AND:
  - true AND unknown = unknown
  - false AND unknown = false
  - unknown AND unknown = unknown
- OR:
  - true OR unknown = true
  - false OR unknown = unknown
  - unknown OR unknown = unknown
- Result of WHERE clause predicate is treated as **false** if it evaluates to **unknown**

## Aggregate Functions

- Just like the ones in RA, these functions operate on the multiset of values of a column of a relation and return a value
  - avg, min, max, sum, count
- Examples:
  - Find the average salary of instructors in the Computer Science department

```
SELECT AVG(salary)  
FROM instructor  
WHERE dept_name = 'Comp. Sci.'
```

- Find the total number of instructors who teach a course in the Spring 2018 semester

```
SELECT COUNT(DISTINCT ID)
FROM teaches
WHERE semester = 'Spring' AND year = 2018
```

- Find the number of tuples in the course relation

```
SELECT COUNT(*)
FROM course
```

## GROUP BY

- Group by: group tuple that have the same values into summary rows
- For example:

Find the average salary of instructors in each department

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
```

| ID    | name       | dept_name  | salary | dept_name  | avg_salary |
|-------|------------|------------|--------|------------|------------|
| 76766 | Crick      | Biology    | 72000  | Biology    | 72000      |
| 45565 | Katz       | Comp. Sci. | 75000  | Comp. Sci. | 77333      |
| 10101 | Srinivasan | Comp. Sci. | 65000  | Elec. Eng. | 80000      |
| 83821 | Brandt     | Comp. Sci. | 92000  | Finance    | 85000      |
| 98345 | Kim        | Elec. Eng. | 80000  | History    | 61000      |
| 12121 | Wu         | Finance    | 90000  | Music      | 40000      |
| 76543 | Singh      | Finance    | 80000  | Physics    | 91000      |
| 32343 | El Said    | History    | 60000  |            |            |
| 58583 | Califeri   | History    | 62000  |            |            |
| 15151 | Mozart     | Music      | 40000  |            |            |
| 33456 | Gold       | Physics    | 87000  |            |            |
| 22222 | Einstein   | Physics    | 95000  |            |            |

- **Syarat:** Attributes in **SELECT** clause outside of aggregate functions must appear in **GROUP BY** list

### WRONG:

```
SELECT person, supervisor
FROM emp_super
GROUP BY person;
```

### CORRECT:

```
SELECT person, supervisor
FROM emp_super
GROUP BY person, supervisor;
```

## HAVING

- Having clause was added because WHERE couldn't be used with aggregate functions
- Example: Find the names and average salaries of all departments whose average salary is greater than 42000

```
SELECT dept_name, AVG(salary) AS avg_salary  
FROM instructor  
GROUP BY dept_name  
HAVING AVG(salary) > 42000;
```

- Predicates in the **HAVING** clause are applied **after** forming groups, whereas predicates in the **WHERE** clause are applied **before** forming groups

## Joined Relations

Join operations take two relations and return as a result another relation.

### Natural Join

- Natural join matches tuples with the same values for all common attributes and retains only one copy of each common column
- For example: List the names of instructors along with the course ID of the courses that they taught

```
SELECT name, course_id  
FROM student, takes  
WHERE student.ID = takes.ID;
```

#### Same query with natural join:

```
SELECT name, course_id  
FROM student NATURAL JOIN takes;  
// can have multiple (>2) relations combined using natural join
```

| tudent relation: |          |            |          | takes relation: |           |        |          |      |       |       | student natural join takes: |            |          |           |        |          |      |       |  |  |
|------------------|----------|------------|----------|-----------------|-----------|--------|----------|------|-------|-------|-----------------------------|------------|----------|-----------|--------|----------|------|-------|--|--|
| ID               | name     | dept_name  | tot_cred | ID              | course_id | sec_id | semester | year | grade | ID    | name                        | dept_name  | tot_cred | course_id | sec_id | semester | year | grade |  |  |
| 00128            | Zhang    | Comp. Sci. | 102      | 00128           | CS-101    | 1      | Fall     | 2017 | A     | 00128 | Zhang                       | Comp. Sci. | 102      | CS-101    | 1      | Fall     | 2017 | A     |  |  |
| 12345            | Shankar  | Comp. Sci. | 32       | 00128           | CS-347    | 1      | Fall     | 2017 | A-    | 00128 | Zhang                       | Comp. Sci. | 102      | CS-347    | 1      | Fall     | 2017 | A-    |  |  |
| 19991            | Brandt   | History    | 80       | 12345           | CS-101    | 1      | Fall     | 2017 | C     | 12345 | Shankar                     | Comp. Sci. | 32       | CS-101    | 1      | Fall     | 2017 | C     |  |  |
| 23121            | Chavez   | Finance    | 110      | 12345           | CS-190    | 2      | Spring   | 2017 | A     | 12345 | Shankar                     | Comp. Sci. | 32       | CS-190    | 2      | Spring   | 2017 | A     |  |  |
| 44553            | Peltier  | Physics    | 56       | 12345           | CS-315    | 1      | Spring   | 2018 | A     | 12345 | Shankar                     | Comp. Sci. | 32       | CS-315    | 1      | Spring   | 2018 | A     |  |  |
| 45678            | Levy     | Physics    | 46       | 12345           | CS-347    | 1      | Fall     | 2017 | A     | 12345 | Shankar                     | Comp. Sci. | 32       | CS-347    | 1      | Fall     | 2017 | A     |  |  |
| 54321            | Williams | Comp. Sci. | 54       | 19991           | HIS-351   | 1      | Spring   | 2018 | B     | 19991 | Brandt                      | History    | 80       | HIS-351   | 1      | Spring   | 2018 | B     |  |  |
| 55739            | Sanchez  | Music      | 38       | 23121           | FIN-201   | 1      | Spring   | 2018 | C+    | 23121 | Chavez                      | Finance    | 110      | FIN-201   | 1      | Spring   | 2018 | C+    |  |  |
| 70557            | Snow     | Physics    | 0        | 44553           | PHY-101   | 1      | Fall     | 2017 | B-    | 44553 | Peltier                     | Physics    | 56       | PHY-101   | 1      | Fall     | 2017 | B-    |  |  |
| 76543            | Brown    | Comp. Sci. | 58       | 45678           | CS-101    | 1      | Fall     | 2017 | F     | 45678 | Levy                        | Physics    | 46       | CS-101    | 1      | Fall     | 2017 | F     |  |  |
| 76653            | Aoi      | Elec. Eng. | 60       | 45678           | CS-101    | 1      | Spring   | 2018 | B+    | 45678 | Levy                        | Physics    | 46       | CS-101    | 1      | Spring   | 2018 | B+    |  |  |
| 98765            | Bourikas | Elec. Eng. | 98       | 45678           | CS-319    | 1      | Spring   | 2018 | B     | 45678 | Levy                        | Physics    | 46       | CS-319    | 1      | Spring   | 2018 | B     |  |  |
| 98988            | Tanaka   | Biology    | 120      | 54321           | CS-101    | 1      | Fall     | 2017 | A-    | 54321 | Williams                    | Comp. Sci. | 54       | CS-101    | 1      | Fall     | 2017 | A-    |  |  |
|                  |          |            |          | 54321           | CS-190    | 2      | Spring   | 2017 | B+    | 54321 | Williams                    | Comp. Sci. | 54       | CS-190    | 2      | Spring   | 2017 | B+    |  |  |
|                  |          |            |          | 55739           | MU-199    | 1      | Spring   | 2018 | A-    | 55739 | Sanchez                     | Music      | 38       | MU-199    | 1      | Spring   | 2018 | A-    |  |  |
|                  |          |            |          | 76543           | CS-101    | 1      | Fall     | 2017 | A     | 76543 | Brown                       | Comp. Sci. | 58       | CS-101    | 1      | Fall     | 2017 | A     |  |  |
|                  |          |            |          | 76543           | CS-319    | 2      | Spring   | 2018 | A     | 76543 | Brown                       | Comp. Sci. | 58       | CS-319    | 2      | Spring   | 2018 | A     |  |  |
|                  |          |            |          | 76653           | EE-181    | 1      | Spring   | 2017 | C     | 76653 | Aoi                         | Elec. Eng. | 60       | EE-181    | 1      | Spring   | 2017 | C     |  |  |
|                  |          |            |          | 98765           | CS-101    | 1      | Fall     | 2017 | C-    | 98765 | Bourikas                    | Elec. Eng. | 98       | CS-101    | 1      | Fall     | 2017 | C-    |  |  |
|                  |          |            |          | 98765           | CS-315    | 1      | Spring   | 2018 | B     | 98765 | Bourikas                    | Elec. Eng. | 98       | CS-315    | 1      | Spring   | 2018 | B     |  |  |
|                  |          |            |          | 98988           | BIO-101   | 1      | Summer   | 2017 | A     | 98988 | Tanaka                      | Biology    | 120      | BIO-101   | 1      | Summer   | 2017 | A     |  |  |
|                  |          |            |          | 98988           | BIO-301   | 1      | Summer   | 2018 | null  | 98988 | Tanaka                      | Biology    | 120      | BIO-301   | 1      | Summer   | 2018 | null  |  |  |

- **Danger of Natural Join:**

- SQL will join EVERY column with the same attribute name.
- Beware of unrelated attributes with the same name which get equated incorrectly.

## Inner Join

- Inner join is used to combine rows from two or more tables based on a related column between them. → acts like **intersection** → selects **only** the **overlapping** data.
- In most SQL databases if you use the JOIN keyword without a prefix, it defaults to an INNER JOIN

## Outer Join

- An extension of the JOIN or INNER JOIN operation that avoids loss of information
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join

Relation *course*

| <i>course_id</i> | <i>title</i> | <i>dept_name</i> | <i>credits</i> |
|------------------|--------------|------------------|----------------|
| BIO-301          | Genetics     | Biology          | 4              |
| CS-190           | Game Design  | Comp. Sci.       | 4              |
| CS-315           | Robotics     | Comp. Sci.       | 3              |

Relation *prereq*

| <i>course_id</i> | <i>prereq_id</i> |
|------------------|------------------|
| BIO-301          | BIO-101          |
| CS-190           | CS-101           |
| CS-347           | CS-101           |

Observe that

*course* information is missing CS-347

*prereq* information is missing CS-315

### Left Outer Join

course NATURAL LEFT OUTER JOIN prereq

| <i>course_id</i> | <i>title</i> | <i>dept_name</i> | <i>credits</i> | <i>prereq_id</i> |
|------------------|--------------|------------------|----------------|------------------|
| BIO-301          | Genetics     | Biology          | 4              | BIO-101          |
| CS-190           | Game Design  | Comp. Sci.       | 4              | CS-101           |
| CS-315           | Robotics     | Comp. Sci.       | 3              | <i>null</i>      |

### Right Outer Join

course NATURAL RIGHT OUTER JOIN prereq

| <i>course_id</i> | <i>title</i> | <i>dept_name</i> | <i>credits</i> | <i>prereq_id</i> |
|------------------|--------------|------------------|----------------|------------------|
| BIO-301          | Genetics     | Biology          | 4              | BIO-101          |
| CS-190           | Game Design  | Comp. Sci.       | 4              | CS-101           |
| CS-347           | <i>null</i>  | <i>null</i>      | <i>null</i>    | CS-101           |

### Full Outer Join

course NATURAL FULL OUTER JOIN prereq

| <i>course_id</i> | <i>title</i> | <i>dept_name</i> | <i>credits</i> | <i>prereq_id</i> |
|------------------|--------------|------------------|----------------|------------------|
| BIO-301          | Genetics     | Biology          | 4              | BIO-101          |
| CS-190           | Game Design  | Comp. Sci.       | 4              | CS-101           |
| CS-315           | Robotics     | Comp. Sci.       | 3              | <i>null</i>      |
| CS-347           | <i>null</i>  | <i>null</i>      | <i>null</i>    | CS-101           |

## Joined Types and Conditions

- Join condition: defines which tuples in the two relations match
  - join type ... on <predicate> using (...)
- Join type: defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated
  - inner join, outer join
- Examples:

- course NATURAL RIGHT OUTER JOIN prereq

| course_id | title       | dept_name  | credits | prereq_id |
|-----------|-------------|------------|---------|-----------|
| BIO-301   | Genetics    | Biology    | 4       | BIO-101   |
| CS-190    | Game Design | Comp. Sci. | 4       | CS-101    |
| CS-347    | null        | null       | null    | CS-101    |

- course FULL OUTER JOIN prereq USING(course\_id)

| course_id | title       | dept_name  | credits | prereq_id |
|-----------|-------------|------------|---------|-----------|
| BIO-301   | Genetics    | Biology    | 4       | BIO-101   |
| CS-190    | Game Design | Comp. Sci. | 4       | CS-101    |
| CS-315    | Robotics    | Comp. Sci. | 3       | null      |
| CS-347    | null        | null       | null    | CS-101    |

- course INNER JOIN prereq ON course.course\_id = prereq.course\_id

| course_id | title       | dept_name  | credits | prereq_id | course_id |
|-----------|-------------|------------|---------|-----------|-----------|
| BIO-301   | Genetics    | Biology    | 4       | BIO-101   | BIO-301   |
| CS-190    | Game Design | Comp. Sci. | 4       | CS-101    | CS-190    |

Notes: no difference with natural join

- course LEFT OUTER JOIN prereq ON course.course\_id = prereq.course\_id

| course_id | title       | dept_name  | credits | prereq_id | course_id |
|-----------|-------------|------------|---------|-----------|-----------|
| BIO-301   | Genetics    | Biology    | 4       | BIO-101   | BIO-301   |
| CS-190    | Game Design | Comp. Sci. | 4       | CS-101    | CS-190    |
| CS-315    | Robotics    | Comp. Sci. | 3       | null      | null      |

## Nested Subqueries

- A subquery is a select-from-where expression that is nested within another query

### Nested Query in WHERE Clause

- **Set Membership (IN)**: to check whether an attribute value is a member of the result set produced by a subquery

- **Set Comparison (<relational operator> SOME/ALL)**: to evaluate whether an attribute value satisfies the <relational operator> condition against some or all tuples in the subquery result
- **Test for Empty Relation (EXISTS)**: to determine whether a subquery returns any tuples
- **Test for Absence of Duplicates (UNIQUE)**: to check whether the tuples in the subquery result are unique (i.e., no duplicate values exist)

### Set Membership

|                                                                                                                    |                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Find courses offered in Fall 2017 and in Spring 2018                                                               | SELECT DISTINCT course_id<br>FROM section<br>WHERE semester = 'Fall' AND year = 2017<br>AND course_id IN (SELECT course_id<br>FROM section<br>WHERE semester = 'Spring'<br>AND year = 2018);     |
| Find courses offered in Fall 2017 but not in Spring 2018                                                           | SELECT DISTINCT course_id<br>FROM section<br>WHERE semester = 'Fall' AND year = 2017<br>AND course_id NOT IN (SELECT course_id<br>FROM section<br>WHERE semester = 'Spring'<br>AND year = 2018); |
| Name all instructors whose name is neither "Mozart" nor "Einstein"                                                 | SELECT DISTINCT name<br>FROM instructor<br>WHERE name NOT IN('Mozart', 'Einstein')                                                                                                               |
| Find the total number of (distinct) students who have taken course sections taught by the instructor with ID 10101 | SELECT COUNT(DISTINCT ID)<br>FROM takes<br>WHERE (course_id, sec_id, semester, year) IN<br>(SELECT course_id, sec_id, semester, year<br>FROM teaches<br>WHERE teaches.ID = 10101);               |



## Set Comparison

### SOME Clause

- $F < \text{comp} > \text{SOME } r \Leftrightarrow \exists t \in r \text{ such that } (F < \text{comp} > t)$ 
  - Where  $< \text{comp} >$  can be  $<, \leq, >, =, \neq$
  - Arti: "ada minimal satu nilai di  $r$  yang membuat perbandingan tsb bernilai TRUE"
- Illustration:

```
(5 < some 

|   |
|---|
| 0 |
| 5 |
| 6 |

) = true      (read: 5 < some tuple in the relation)

(5 < some 

|   |
|---|
| 0 |
| 5 |

) = false

(5 = some 

|   |
|---|
| 0 |
| 5 |

) = true

(5 ≠ some 

|   |
|---|
| 0 |
| 5 |

) = true (since 0 ≠ 5)

(= some) ≡ in
However, (≠ some) ≡ not in
```

- Example:  
Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department

|                     |                                                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------------------------------|
| without SOME clause | SELECT DISTINCT T.name<br>FROM instructor AS T, instructor AS S<br>WHERE T.salary > S.salary AND s.dept_name = 'Biology'; |
| with SOME clause    | SELECT name<br>FROM instructor<br>WHERE salary > some (SELECT salary<br>FROM instructor<br>WHERE dept_name = 'Biology');  |

## ALL Clause

- $F < \text{comp} > \text{ALL } r \Leftrightarrow \forall t \in r (F < \text{comp} > t)$
- Illustration:

|                                                                                      |                                                                          |   |    |                                                      |                    |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------|---|----|------------------------------------------------------|--------------------|
| $(5 < \text{all}$                                                                    | <table><tr><td>0</td></tr><tr><td>5</td></tr><tr><td>6</td></tr></table> | 0 | 5  | 6                                                    | $) = \text{false}$ |
| 0                                                                                    |                                                                          |   |    |                                                      |                    |
| 5                                                                                    |                                                                          |   |    |                                                      |                    |
| 6                                                                                    |                                                                          |   |    |                                                      |                    |
| $(5 < \text{all}$                                                                    | <table><tr><td>6</td></tr><tr><td>10</td></tr></table>                   | 6 | 10 | $) = \text{true}$                                    |                    |
| 6                                                                                    |                                                                          |   |    |                                                      |                    |
| 10                                                                                   |                                                                          |   |    |                                                      |                    |
| $(5 = \text{all}$                                                                    | <table><tr><td>4</td></tr><tr><td>5</td></tr></table>                    | 4 | 5  | $) = \text{false}$                                   |                    |
| 4                                                                                    |                                                                          |   |    |                                                      |                    |
| 5                                                                                    |                                                                          |   |    |                                                      |                    |
| $(5 \neq \text{all}$                                                                 | <table><tr><td>4</td></tr><tr><td>6</td></tr></table>                    | 4 | 6  | $) = \text{true}$ (since $5 \neq 4$ and $5 \neq 6$ ) |                    |
| 4                                                                                    |                                                                          |   |    |                                                      |                    |
| 6                                                                                    |                                                                          |   |    |                                                      |                    |
| $(\neq \text{all}) \equiv \text{not in}$<br>However, $(= \text{all}) \neq \text{in}$ |                                                                          |   |    |                                                      |                    |

- Example: (same problem as before)  
Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department

```
SELECT name
FROM instructor
WHERE salary > ALL (SELECT salary
                    FROM instructor
                    WHERE dept_name = 'Biology');
```

## Test for Empty Relations

- EXISTS returns TRUE if the argument subquery is not empty
- $\text{EXISTS } r \Leftrightarrow r \neq \emptyset$
- $\text{NOT EXISTS } r \Leftrightarrow r = \emptyset$

## EXISTS Clause

- Example: Find all courses taught in both Fall 2017 and in the Spring 2018

```
SELECT course_id
FROM section AS S
WHERE semester = 'Fall' AND year = 2017
AND EXISTS (SELECT *
            FROM section AS T
            WHERE semester = 'Spring' AND year = 2018
            AND S.course_id = T.course_id);
```

- Correlation name = var. S in the outer query
- Correlated subquery = the inner query

### NOT EXISTS Clause

- Example: Find all students who have taken all courses offered in the Biology department

```
SELECT DISTINCT S.ID, S.name
FROM student AS S
WHERE NOT EXISTS(
    (SELECT course_id FROM course WHERE dept_name = 'Biology')
    \\\semua course biology
    EXCEPT \\\kecuali
    (SELECT T.course_id FROM takes AS T WHERE S.ID = T.ID)
    \\\course yang diambil mahasiswa S

    \\\arti: semua course biology yang belum diambil mahasiswa S
); \\\ where not exists → tidak ada course yang belum diambil mahasiswa S
```

- $X - Y = \emptyset \Leftrightarrow X \subseteq Y$
- Tidak bisa pakai ALL karena ALL adalah perbandingan NILAI (bandingkan satu nilai dengan banyak nilai), bukan set coverage (kasus ini adalah perbandingan antar set)
- Tidak pakai NOT IN karena jika subquery mengandung NULL, maka hasilnya akan selalu FALSE ( $x \neq \text{NULL} \rightarrow \text{UNKNOWN} \rightarrow \text{always evaluates to FALSE}$ )

### Test for Absence of Duplicate Tuples

- UNIQUE tests whether a subquery has any duplicate tuples in its results
  - TRUE if contains no duplicates
  - FALSE if otherwise
- Example: Find all courses that were offered at most once in 2017

```
SELECT T.course_id
FROM course AS T
WHERE UNIQUE( SELECT R.course_id
              FROM section AS R
              WHERE T.course_id = R.course_id
              AND R.year = 2017 );
```

## Nested Queries in the FROM Clause

Example:

|                                                                                                               |                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Find the average instructors' salaries of those departments where the average salary is greater than \$42,000 | SELECT dept_name, avg_salary<br>FROM (SELECT dept_name, AVG(salary) AS avg_salary<br>FROM instructor<br>GROUP BY dept_name)<br>WHERE avg_salary > 42000;                      |
|                                                                                                               | SELECT dept_name, avg_salary<br>FROM (SELECT dept_name, AVG(salary)<br>FROM instructor<br>GROUP BY dept_name) AS dept_avg(dept_name, avg_salary)<br>WHERE avg_salary > 42000; |

## WITH Clause

- Provides a way of defining a temporary relation whose definition is available only to the query in which the WITH clause occurs
- Example:

- Find all departments with the maximum budget

```
WITH max_budget(value) AS (  
    SELECT MAX(budget)  
    FROM department  
)  
  
SELECT department_name  
FROM department, max_budget  
WHERE department.budget = max_budget.value;
```

- Find all departments where the total salary is greater than the average of the total salary at all departments

```
WITH dept_total(dept_name, value) AS (  
    SELECT dept_name, SUM(salary)  
    FROM instructor  
    GROUP BY dept_name),  
  
dept_total_avg(value) AS (  
    SELECT AVG(value)  
    FROM dept_total)
```

```

        SELECT AVG(value) FROM dept_total
    )

    SELECT dept_name
    FROM dept_total, dept_total_avg
    WHERE dept_total.value > dept_total_avg.value;

```

## Scalar Subquery

- used when a single value is expected
- Example:

List all departments along with the number of instructors in each department

```

SELECT dept_name, (SELECT COUNT(*)
                    FROM instructor
                    WHERE department.dept_name = instructor.dept_name)
                AS num_instructors
FROM department;

```

- Runtime error if subquery returns **more than one result tuple**

## VIEW

- View provides a mechanism to hide certain data from the view of certain users
- A view is defined using the CREATE VIEW statement, which has the form:

```
CREATE VIEW v AS <query expression>
```

- <query expression> is any legal SQL expression
  - v represents view name
- View definition is not the same as creating a new relation by evaluating the query expression, BUT it is the same as saving an expression, which can be substituted into queries using the view.
- Usage example:

### **A view of instructors without their salary**

```

CREATE VIEW faculty AS
    SELECT ID, name, dept_name
    FROM instructor

```

|                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Find all instructors in the Biology dept.</b>                                                                                                        |
| <pre>SELECT name FROM faculty WHERE dept_name = 'Biology'</pre>                                                                                         |
| <b>Create a view of department salary totals</b>                                                                                                        |
| <pre>CREATE VIEW department_total_salary(dept_name, total_salary) AS     SELECT dept_name, SUM(salary)     FROM instructor     GROUP BY dept_name</pre> |

## View Defined using Other Views

- A view can be used in defining another view
  - A view relation  $v_1$  is said to **depend directly** on a view relation  $v_2$  if  $v_2$  is used in the expression defining  $v_1$
  - A view relation  $v_1$  is said to **depend on** view relation  $v_2$  if either  $v_1$  depends directly to  $v_2$  or there is a path of dependencies from  $v_1$  to  $v_2$
  - A view relation  $v$  is said to be **recursive** if it depends on itself
- Example:

```
CREATE VIEW physics_fall_2017 AS
    SELECT course.course_id, sec_id, building, room_number
    FROM course, section
    WHERE course.course_id = section.course_id
    AND course.dept_name = 'Physics'
    AND section.semester = 'Fall'
    AND section.year = 2017;

CREATE VIEW physics_fall_2017_watson AS
    SELECT course_id, room_number
    FROM physics_fall_2017
    WHERE building = 'Watson';
```

## Materialized Views

- Upon defining a view, a physical copy is created → materialized view

- If relation used in the query are updated, the materialized view result becomes out of date → need to maintain the view by updating the view whenever the underlying relations are updated.

## Modification of the Database

### Deletion

```
delete from <table_name>
where <condition>;
```

Examples:

|                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Delete all instructors                                                                                          | DELETE FROM instructor                                                                                                                                                                                                                                                                                                                         |
| Delete all instructors from the Finance dept.                                                                   | DELETE FROM instructor<br>WHERE dept_name = 'Finance'                                                                                                                                                                                                                                                                                          |
| Delete all tuples in the instructor relation for instructors associated with a dept. located in Watson building | DELETE FROM instructor<br>WHERE dept_name IN (SELECT dept_name<br>FROM department<br>WHERE building = 'Watson');                                                                                                                                                                                                                               |
| Delete all instructors whose salary is less than the average salary of instructors                              | DELETE FROM instructor<br>WHERE salary < (SELECT AVG(salary) FROM instructor);<br><br><b>Problem:</b><br>as we delete tuples from instructor, the average salary changes<br><br><b>Solution:</b><br>1. Compute AVG(salary) and find all tuples to delete<br>2. Delete all tuples found above (without recomputing AVG or retesting the tuples) |

## Insertion

```
insert into <table_name> [(<attributes_list>)]  
values (<attributes_values>);
```

Examples:

|                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Add a new tuple to course                                                                                                                           | INSERT INTO course<br>VALUES ('CS-347', 'Database Systems', 'Comp. Sci.', 4);                                                                                                                                                                                                                                              |
|                                                                                                                                                     | INSERT INTO course (course_id, title, dept_name, credits)<br>VALUES ('CS-347', 'Database Systems', 'Comp. Sci.', 4);                                                                                                                                                                                                       |
| Add a new tuple to student with tot_creds set to null                                                                                               | INSERT INTO student<br>VALUES ('3003', 'Green', 'Finance', null);                                                                                                                                                                                                                                                          |
| Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of \$18,000 | INSERT INTO instructor<br>SELECT ID, name, dept_name, 18000<br>FROM student<br>WHERE dept_name = 'Music' AND total_cred > 144<br><br>The select from where statement is evaluated fully before any of its results are inserted. Otherwise, queries like:<br>INSERT INTO table1 SELECT * FROM table1<br>would cause problem |

## Updating

```
update <table_name>  
set <attribute_name> = <expression>  
where <condition>;
```

Examples:

|                                                                      |                                                                        |
|----------------------------------------------------------------------|------------------------------------------------------------------------|
| Give a 5% salary raise to all instructors                            | UPDATE instructor<br>SET salary = salary *1.05;                        |
| Give a 5% salary raise to those instructors who earn less than 70000 | UPDATE instructor<br>SET salary = salary*1.05<br>WHERE salary < 70000; |
| Give a 5% salary raise to instructors whose                          | UPDATE instructor<br>SET salary = salary*1.05                          |



|                                                                                             |                                                                                                                                                       |
|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| salary is less than average                                                                 | WHERE salary < (SELECT AVG(salary) FROM instructor);                                                                                                  |
| Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by 5% | UPDATE instructor<br>SET salary = salary*1.03<br>WHERE salary > 100000<br><br>UPDATE instructor<br>SET salary = salary*1.05<br>WHERE salary <= 100000 |

- **Order is important**
- Can be done better using the CASE statement

### CASE Statement for Conditional Updates

Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by 5%

|                        |                                                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Without CASE statement | UPDATE instructor<br>SET salary = salary*1.03<br>WHERE salary > 100000<br><br>UPDATE instructor<br>SET salary = salary*1.05<br>WHERE salary <= 100000 |
| With CASE statement    | UPDATE instructor<br>SET salary = CASE<br>WHEN salary <= 100000 THEN salary*1.05<br>ELSE salary*1.03<br><br>END                                       |

### Updates with Scalar Subqueries

Recompute and update tot\_creds value for all students

```
UPDATE student S
SET tot_creds = (SELECT SUM(credits)
                FROM takes, course
                WHERE takes.course_id = course.course_id
                AND S.ID = takes.ID
                AND takes.grade <> 'F'
                AND takes.grade IS NOT NULL);
```

→ sets tot\_creds to null for students who have not taken any course.  
→ instead of this, we can use:

```
UPDATE student S
SET tot_creds = CASE
                WHEN SUM(credits) IS NOT NULL THEN SUM(credits) ELSE 0
END
```

### Update of a View

Add a new tuple to faculty view which we defined earlier

```
INSERT INTO faculty
VALUES ('30765', 'Green', 'Music');
```

The insertion must be represented by the insertion into the instructor relation. So, there are two approaches:

- reject the insert
- insert the tuple ('30765', 'Green', 'Music', null) into the instructor relation

Most SQL implementations allow updates only on simple views

- The from clause has only one database relation.
- The select clause contains only attribute names of the relation, and does not have any expressions, aggregates, or distinct specification.
- Any attribute not listed in the select clause can be set to null
- The query does not have a group by or having clause.

## Data Definition Language

- The SQL DDL allows the specification of information about relations including:
  - The schema for each relation
  - The type of values associated with each attribute
  - The integrity constraints
  - The set of indices to be maintained for each relation
  - Security and authorization information for each relation
  - The physical storage structure of each relation on disk

### Domain Types

|            |                                                                     |
|------------|---------------------------------------------------------------------|
| char(n).   | Fixed length character string, with user-specified length n         |
| varhar(n). | Variable length character strings, with user-specified max length n |
| int.       | Integer (a finite subset of the integers that is machine-dependent) |

|                         |                                                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| smallint.               | Small integer (a machine-dependent subset of the integer dom. type)                                                                                                                                                                          |
| numeric(p,d).           | Fixed point number, with user-specified precision of p digits, with d digits to the right of decimal point. For example, numeric(3,1) allows 44.5 to be stored exactly but not 444.5 or 0.32                                                 |
| real, double precision. | Floating point and double-precision floating point numbers, with machine-dependent precision                                                                                                                                                 |
| float(n).               | Floating point number, with user-specified precision of at least n digits                                                                                                                                                                    |
| date                    | Dates, containing a (4 digit) year, month, and date. For example: date '2005-7-27'                                                                                                                                                           |
| time                    | Time of day, in hours, minutes, and seconds. For example: time '09:00:30'                                                                                                                                                                    |
| timestamp               | date plus time of day. For example: timestamp '2005-7-27 09:00:30.75'                                                                                                                                                                        |
| interval                | period of time. example: interval '1' day <ul style="list-style-type: none"> <li>Subtracting a date/time/timestamp value from another gives an interval value</li> <li>Interval values can be added to date/time/timestamp values</li> </ul> |

## CREATE TABLE

```
CREATE TABLE r
    ( $A_1 D_1, A_2 D_2, \dots, A_n D_n,$ 
     (integrityconstraint1),
     ...,
     (integrityconstraintk))
```

- r is the name of the relation
- each  $A_i$  is an attribute name in the schema of relation r
- $D_i$  is the data type of values in the domain of attribute  $A_i$

## Integrity Constraints

Types of integrity constraints:

- primary key ( $A_1, \dots, A_n$ )
- foreign key ( $A_m, \dots, A_n$ ) references  $r$
- not null

SQL prevents any update to the database that violates an integrity constraint.

Examples:

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    salary      numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references department);
```

```
create table student (  
    ID          varchar(5),  
    name        varchar(20) not null,  
    dept_name   varchar(20),  
    tot_cred    numeric(3,0),  
    primary key (ID),  
    foreign key (dept_name) references department);  
  
create table takes (  
    ID          varchar(5),  
    course_id   varchar(8),  
    sec_id      varchar(8),  
    semester    varchar(6),  
    year        numeric(4,0),  
    grade       varchar(2),  
    primary key (ID, course_id, sec_id, semester, year) ,  
    foreign key (ID) references student,  
    foreign key (course_id, sec_id, semester, year) references section);
```

```
create table course (  
    course_id   varchar(8),  
    title       varchar(50),  
    dept_name   varchar(20),  
    credits     numeric(2,0),  
    primary key (course_id),  
    foreign key (dept_name) references department);
```

## Updates to Tables

### Drop Table

```
DROP TABLE r
```

### Alter

```
ALTER TABLE r ADD A D
```

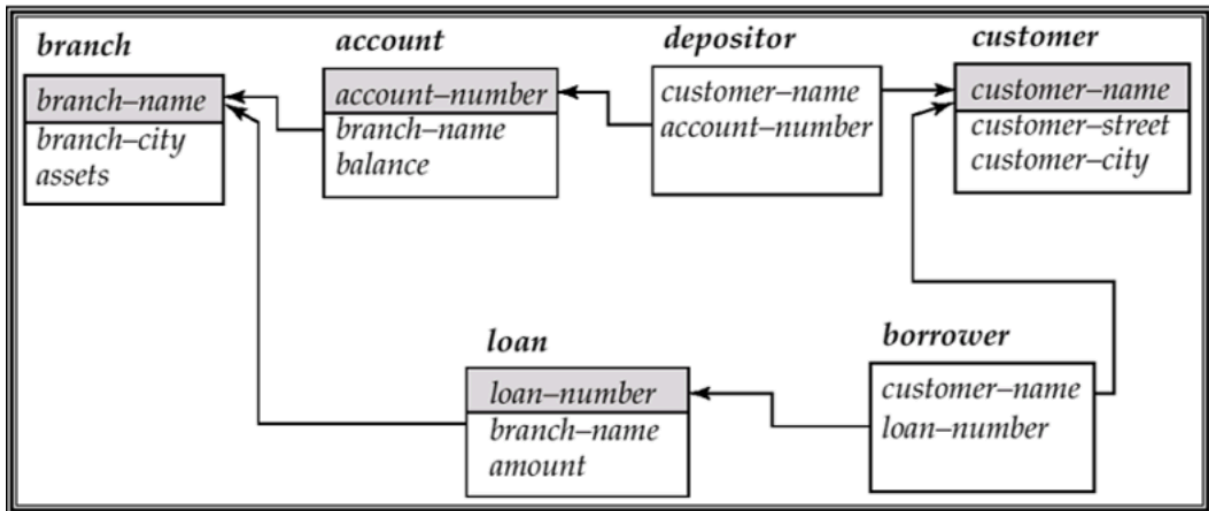
- A is the name of the attribute to be added to relation r
- D is the domain of A
- All exiting tuples in the relation are assigned null as the value for the new attribute

```
ALTER TABLE r DROP A
```

- A is the name of an attribute of relation r
- Dropping of attributes not supported by many databases

## Latihan

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan 2 perintah SQL yang berbeda untuk menjawab persoalan di bawah ini.

Tampilkan data nomor akun, nama cabang, aset cabang, dan saldo akun dari akun dengan saldo di atas \$500 serta berada di cabang dengan nilai aset di atas \$1,000,000 dan nama cabang diakhiri huruf 'k'.

### Perintah SQL pertama:

```
SELECT a.account_number, a.branch_name, b.assets, a.balance
```

```
FROM account a, branch b
```

```
WHERE a.branch_name = b.branch_name
```

```
AND b.assets > 1000000
```

```
AND a.balance > 500
```

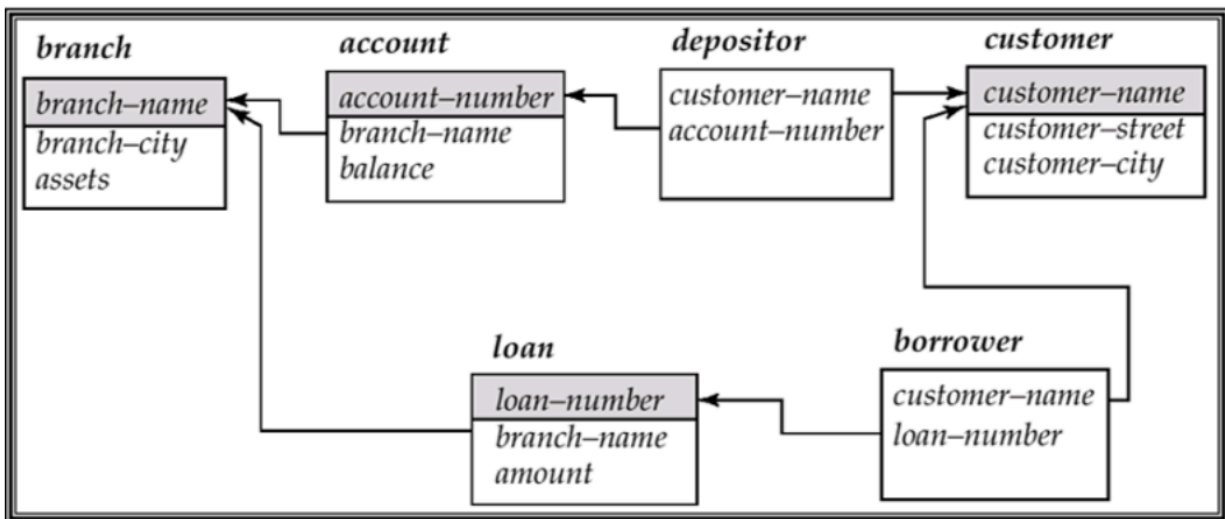
```
AND b.branch_name LIKE "%k";
```

### Perintah SQL kedua:

```
SELECT
```

```
a.account_number,  
  
a.branch_name,  
  
(SELECT b.assets FROM branch b WHERE b.branch_name = a.branch_name) AS assets,  
  
a.balance  
  
FROM account a  
  
WHERE a.balance > 500  
  
AND a.branch_name IN (  
  
    SELECT branch_name  
  
    FROM branch  
  
    WHERE assets > 1000000  
  
    AND branch_name LIKE '%k'  
  
);
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

Tampilkan data nomor akun dan perkiraan nilai saldo yang baru (dengan nama column 'pred\_balance') dari seluruh akun di cabang 'Uptown' jika akun dengan saldo minimum \$5.000 diberikan bonus sebesar 1%.

```
SELECT account_number, balance*1.01 AS pred_balance
```

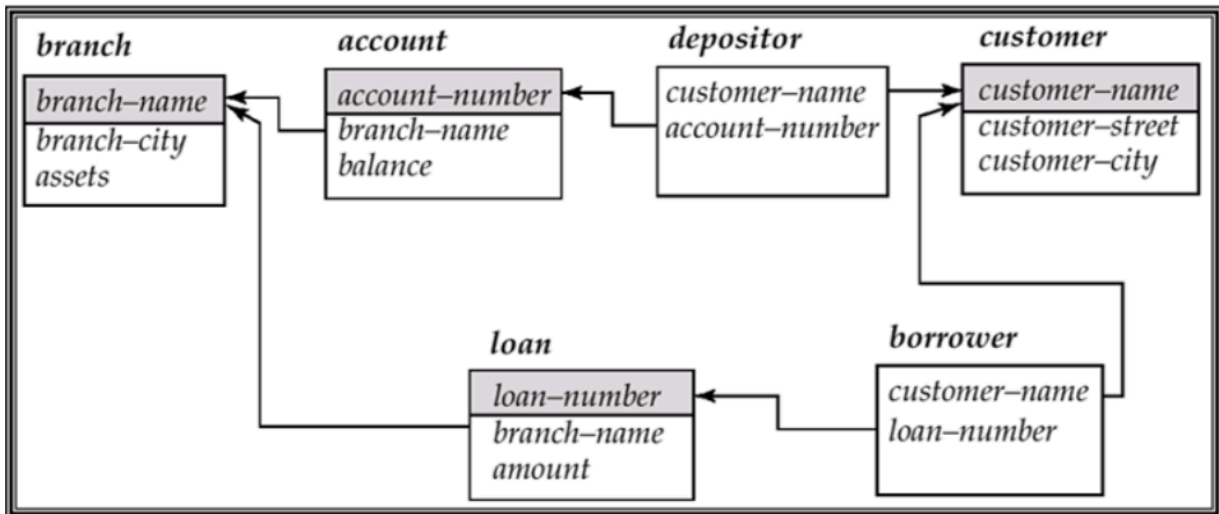
```
FROM account
```

```
WHERE branch_name = "Uptown"
```

```
AND balance ≥ 5000;
```



Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



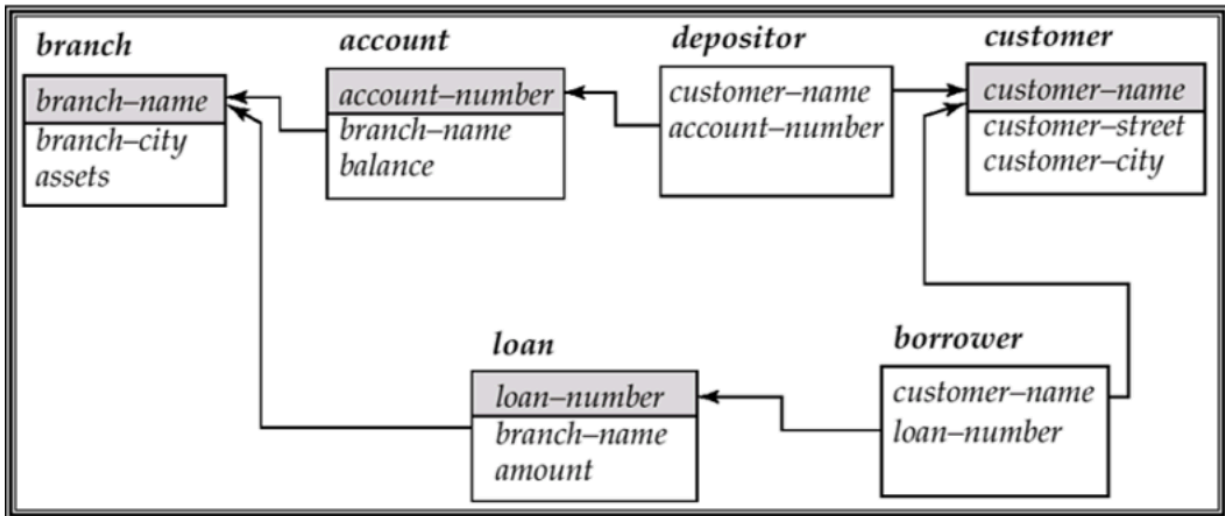
Skema Bank

Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

Tampilkan data nama pelanggan, nomor akun, nilai saldo, kota domisili pelanggan diurutkan berdasarkan nilai saldo dari nilai terbesar ke terkecil. Kemudian untuk akun dengan nilai saldo yang sama diurutkan berdasarkan nomor akun dan nama pelanggan.

```
SELECT c.customer_name, a.account_number, a.balance, c.customer_city
FROM customer c, account a, depositor d
WHERE c.customer_name = d.customer_name
AND a.account_number = d.account_number
ORDER BY a.balance DESC, a.account_number, c.customer_name;
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

Tampilkan data nama cabang, nilai pinjaman terkecil, nilai pinjaman terbesar, nilai rata-rata pinjaman di cabang tsb, jumlah nilai total pinjaman di cabang tsb, dan banyaknya pinjaman yang dikeluarkan di cabang tsb. Yang perlu ditampilkan hanya cabang dengan banyaknya pinjaman di cabang tsb minimal dua.

```

SELECT branch_name,

        MAX(amount) AS max_loan,

        MIN(amount) AS min_loan,

        AVG(amount) AS avg_loan,

        SUM(amount) AS total_loan,

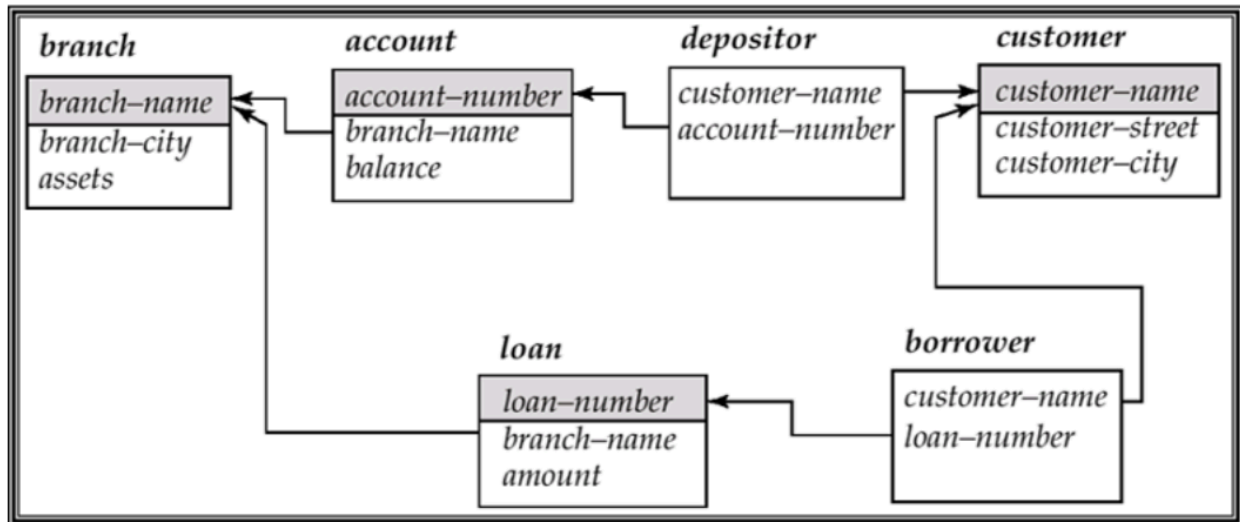
        COUNT(loan_number) AS count_loan

FROM loan

GROUP BY branch_name

HAVING COUNT(loan_number) ≥ 2;
    
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



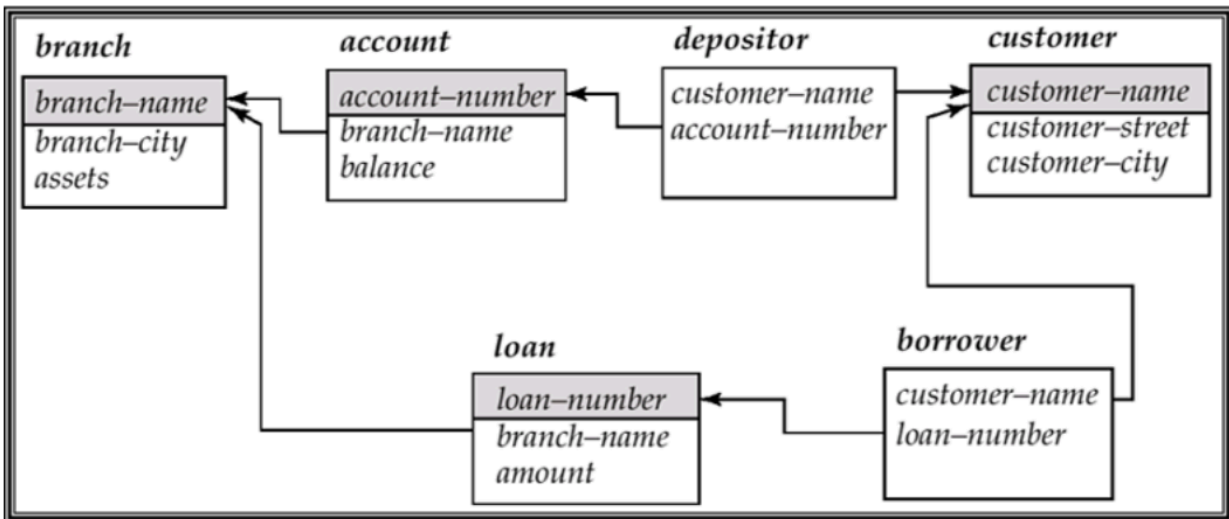
Skema Bank

Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

Tampilkan di kota mana saja customer yang memiliki loan dengan nilai lebih besar dari \$1000 tinggal.

```
SELECT c.customer_city
FROM customer c, borrower b, loan l
WHERE c.customer_name = b.customer_name
AND b.loan_number = l.loan_number
AND l.amount > 1000;
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

Tampilkan pasangan nama cabang bank (branch\_name1, branch\_name2) yang berlokasi di kota yang sama, dan paling tidak salah satunya memiliki aset di atas 2.000.000.000. Pastikan setiap pasangan hanya muncul satu kali.

```
SELECT b1.branch_name AS branch_name1, b2.branch_name AS branch_name2
```

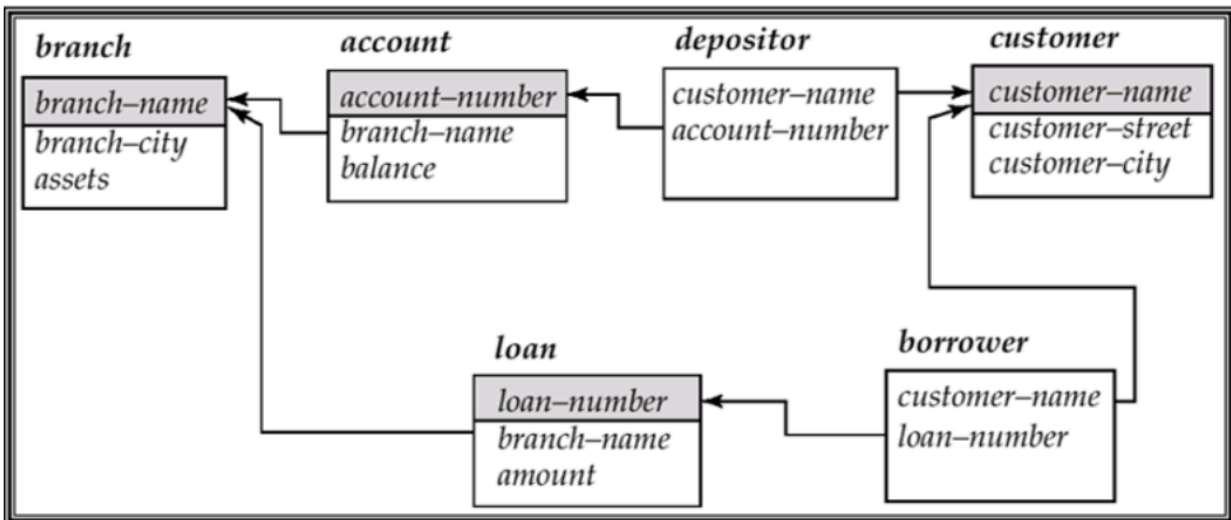
```
FROM branch b1, branch b2
```

```
WHERE b1.branch_city = b2.branch_city
```

```
AND b1.branch_name < b2.branch_name
```

```
AND (b1.assets > 2000000000 OR b2.assets > 2000000000);
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan 2 perintah SQL yang berbeda untuk menjawab persoalan di bawah ini.

Tampilkan daftar nama customer yang selain memiliki simpanan (account) juga memiliki pinjaman (loan) pada cabang bank yang sama. Tampilkan juga di cabang mana customer tersebut memiliki simpanan dan pinjaman tersebut.

```

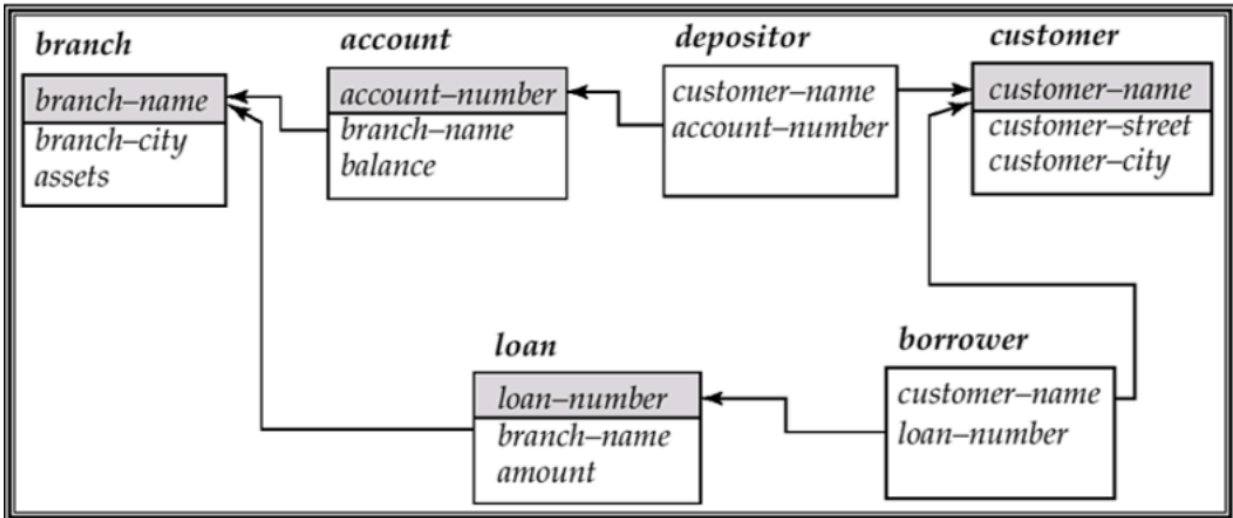
SELECT DISTINCT d.customer_name, a.branch_name
FROM depositor d
JOIN account a
    ON d.account_number = a.account_number
JOIN borrower b
    ON d.customer_name = b.customer_name
JOIN loan l
    ON b.loan_number = l.loan_number
WHERE a.branch_name = l.branch_name;
    
```

```

SELECT DISTINCT d.customer_name, a.branch_name
FROM depositor d, account a, borrower b, loan l
WHERE d.account_number = a.account_number
    
```

AND d.customer\_name = b.customer\_name  
 AND b.loan\_number = l.loan\_number  
 AND a.branch\_name = l.branch\_name;

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

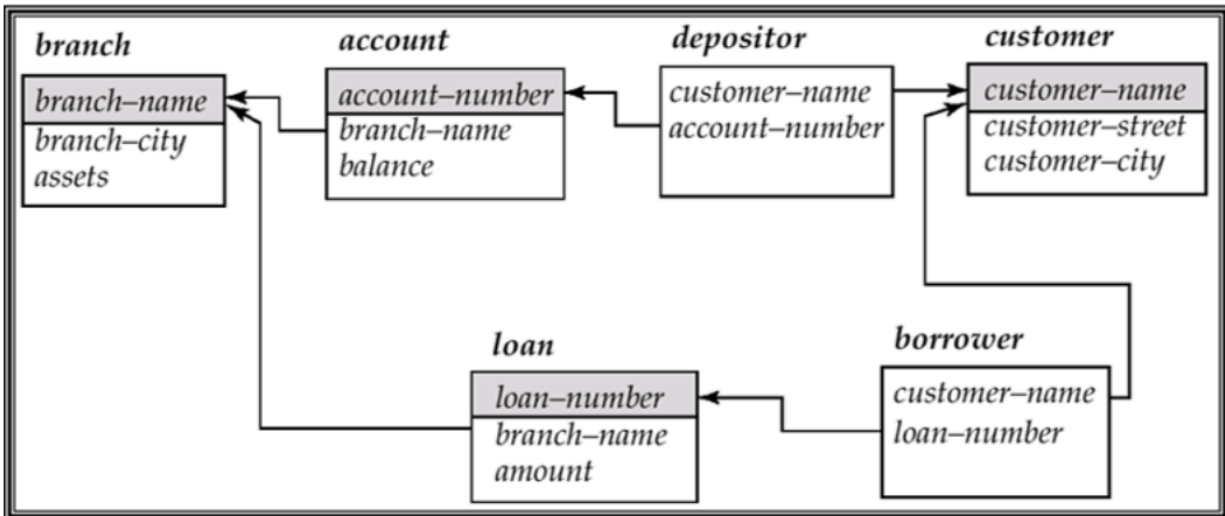
Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini:

Tampilkan daftar nama cabang yang berada di kota 'Brooklyn' yang memiliki total pinjaman di atas 5.000.000.000 namun asetnya di bawah 5.000.000.000.

```

SELECT branch_name
FROM branch
WHERE branch_city = 'Brooklyn'
AND assets < 5000000000
AND branch_name IN (
    SELECT branch_name
    FROM loan
    GROUP BY branch_name
    HAVING SUM(amount) > 5000000000
);
  
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan 3 perintah SQL yang berbeda untuk menjawab persoalan di bawah ini.

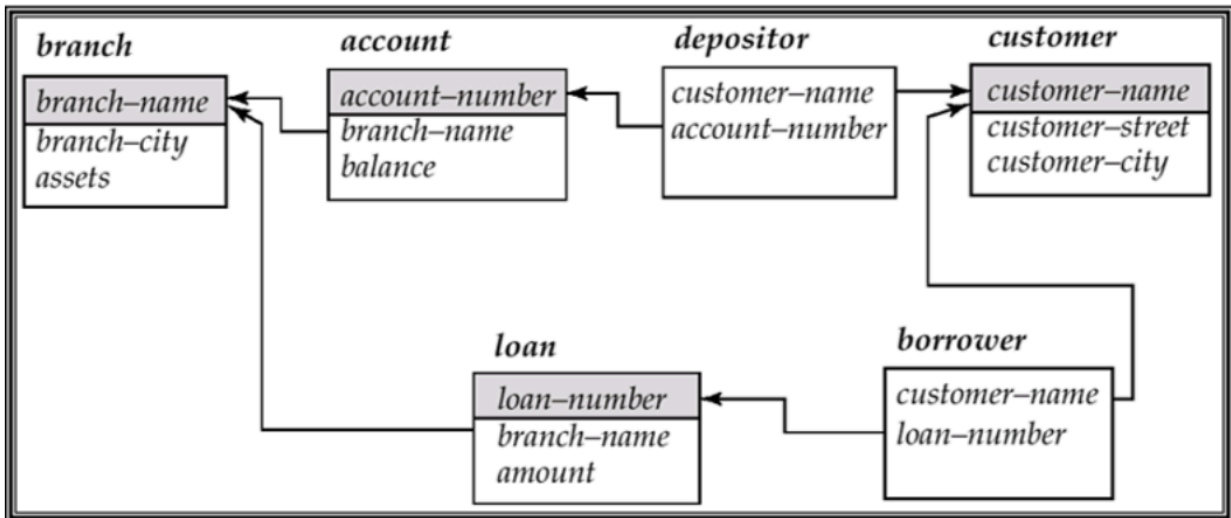
Tampilkan daftar nama customer yang tidak memiliki pinjaman di bank.

```
SELECT customer_name
FROM customer
WHERE customer_name NOT IN (
    SELECT customer_name
    FROM borrower
);
```

```
SELECT customer_name FROM customer
EXCEPT
SELECT customer_name FROM borrower;
```

```
SELECT c.customer_name
FROM customer c
LEFT JOIN borrower b ON c.customer_name = b.customer_name
WHERE b.customer_name IS NULL;
```

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Skema Bank

Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

Tampilkan data pelanggan (nama, alamat, dan kota) yang memiliki akun dengan saldo tertinggi.

```

SELECT customer_name, customer_street, customer_city
FROM customer
WHERE customer_name IN (
    SELECT customer_name
    FROM depositor
    WHERE account_number IN (
        SELECT account_number
        FROM account
        WHERE balance = (SELECT MAX(balance) FROM account)
    )
);
    
```